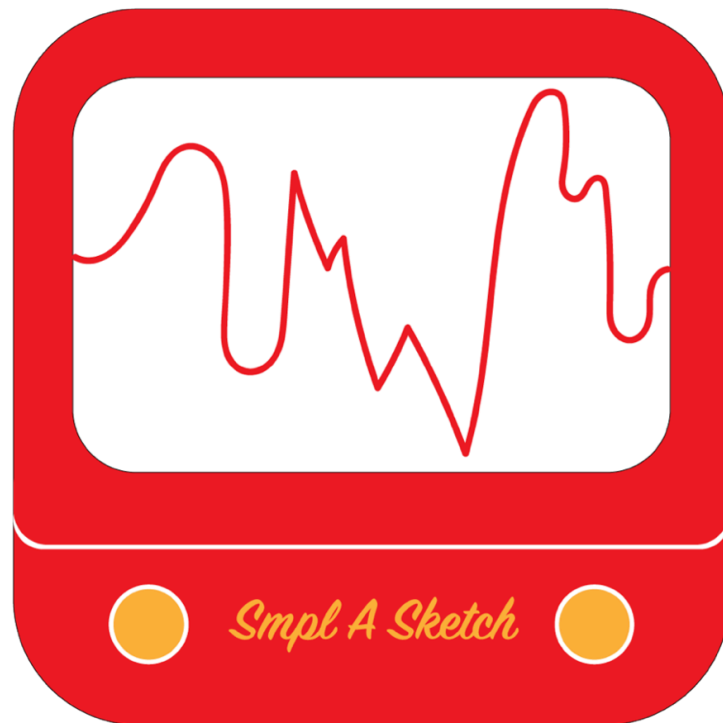

Electronic Product Design Midterm Report

Braden Cantor-Goldner



10/15/2025

What Is It?

The SmplASketch is a draw your own wavetable synth device taking inspiration from the much larger Oramics machines. The physical device has a slot for plexiglass slides to be inserted containing user drawn soundwaves (and eventually LFOs) and the companion desktop app is able to use the data from the slides to create samples. The physical device also allows you to customize pitch, envelope length, and in software there is much more additional customization.

Demo Video: <https://youtu.be/m2uSW2j2sG0>

Documentation

Progress Checklist: [Checklist](#)

Images of the work described in the summaries below are included in the slides for that week for reference.

Week 1: [Slides](#) **Summary:** I wrote the **image processing code** in which the whole project is based off of and I spent a number of hours setting up a headless boot of a **Raspberry Pi** which was the original way I was going to run the code for the project before developing the desktop app.

Week 2: [Slides](#) **Summary:** More **Raspberry Pi work** that eventually got scrapped. Code wise I wrote the **audio sample generation** code and the **envelope generation** code. Added **camera connectivity** code to interface with the video and image libraries.

Week 3: [Slides](#) **Summary:** **Ditched the Raspberry Pi** and built the device on a breadboard with the **Teensy**/fully committed to **desktop app**. Wrote all of the **C++ code** to send extended ASCII characters to control the desktop app. Wrote reset code, data mapping code, ASCII reception code, and display mode code. Designed the entire **primary UI** window and **designed graphics** for the app and app icon.

Week 4: [Slides](#) **Summary:** Designed/ordered **PCB**, designed **V1 slide**, designed **V1 housing**, took apart the **webcam**, and ordered **plexiglass**. Coded Envelope menu, help menu, and test version of **Photosynthesis mode**.

Week 5: [Slides](#) **Summary:** **3D printed V1** of the device and then spent the rest of the week redesigning the device for V2 and **modifying the code** base to work with the device. Also wrote the code for the **waveform editor**.

Week 6: [Slides](#) **Summary:** **3D printed V2** of the device and assembled everything. Tested and **assembled PCB**. **Finished all remaining code** for Version 1 of the app.

What's Left

Fabrication:

From here out the internal fabrication I'm happy with. It's now just a matter of eventually **reprinting** in a different color and getting the **laser cut housing**. I already have my PCB completed so the only real fabrication step left is to design my laser cut housing for the internal 3D printed stuff. I have already tested and mostly put together the PCB so that shouldn't take much more time. I'm just waiting on some additional keyboard switches to arrive. The dimensions for fabrication have already been worked out and are included on the diagram below. In terms of everything fitting inside of the enclosure the keyboard buttons are all soldered to the PCB and there are 2 empty pins on the PCB in which I soldered 2 male to female wires to connect to the LED backlight in the bottom. The camera sits inside the internal 3D printed housing and is connected into the same USB hub output that the Teensy chord is. My backup if the hub causes trouble is just to have two wires coming out of the device. I **may 3D print custom key switches** but that's just an extra if I have time as I am fine with standard keycaps.

Code:

Code wise this is what I see myself spending the rest of the semester working on. I happy-ish with V1 of the software but I think it could use a whole UI rework and while the internal code I probably won't touch I will need to **rewrite the UI from scratch**. My big issues with it right now are the side menus being pop up windows and that I designed this UI back when I thought the second capture slot on the device would be for an envelope not for an LFO. I want to redesign the UI to look like eclipse or some other IDE where there's the primary center window with menus permanently all around the center window. The other code thing I want to work on other than the UI and **adding an LFO mode** is to make the **photosynthesis mode** do really cool stuff. After all of that I just need to **sign the application** so that other people can download it.

Detailed To-Do List:

Week 7:

This week is primarily going to be focused on teaching myself how to do the **laser cutting** and then creating a final design for the outer housing for my device. Secondly I want to start writing the **new UI code** by just getting the different menus to sit in the right place. I'll do this in an entirely separate code base from the internal stuff for the time being. I also want to order the internal USB hub for cable management and the mountable USB C port.

Week 8:

The primary goal of this week is **fabricating the laser cut housing** and putting it together. Code wise I want to **connect all of the internal code** that I have to the new UI and **design new graphics** to give the software some depth.

Week 9:

No fabrication week, just spend the whole week writing the code to get **LFO mode** to work and programming the **tonehead animation** and playback system to look almost DAW like.

Week 10:

Reprint all of the 3D prints in the correct color and construct the final physical version of the device. Write **pitch detection code** to be able to have the arrow keys move the pitch chromatically. I can truly expand the capabilities of Photosynthesis mode.

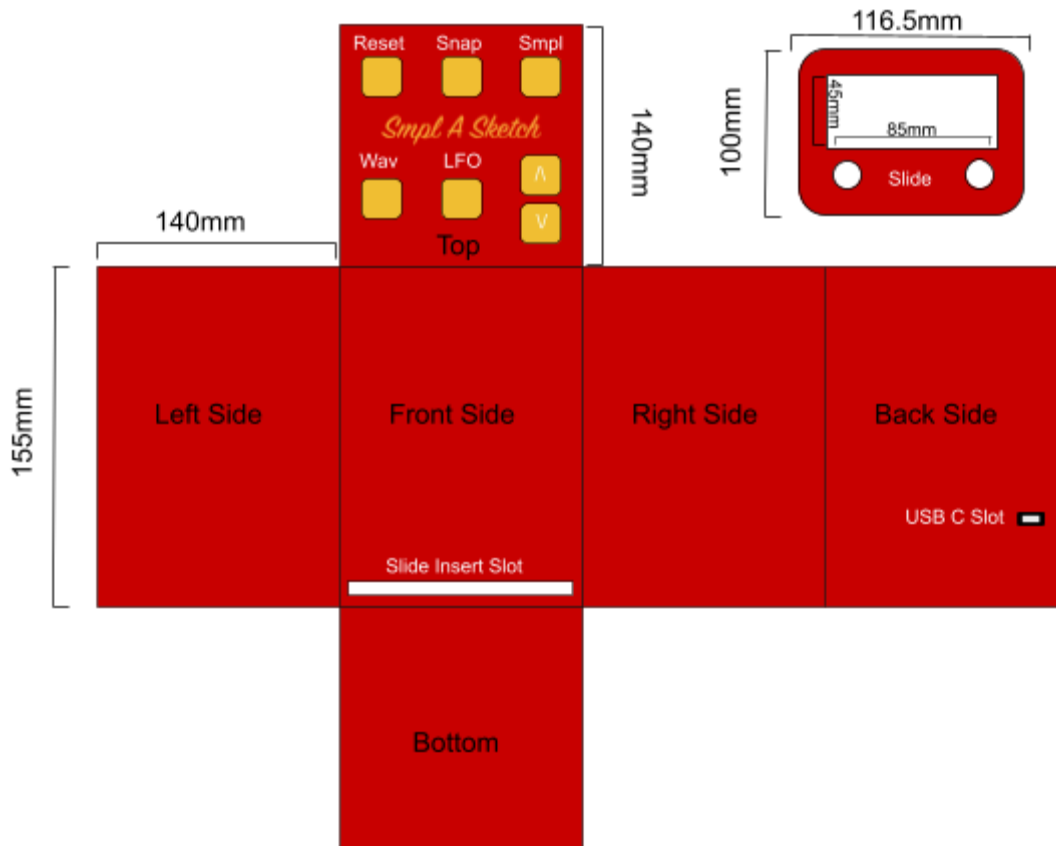
Week 11:

Write the **file IO UI for Photosynthesis mode**(may not be possible with the UI library I'm using). **Sign the application** so that it can be downloaded on other devices. **Photograph the device** and **film a demo** of it working.

Week 12:

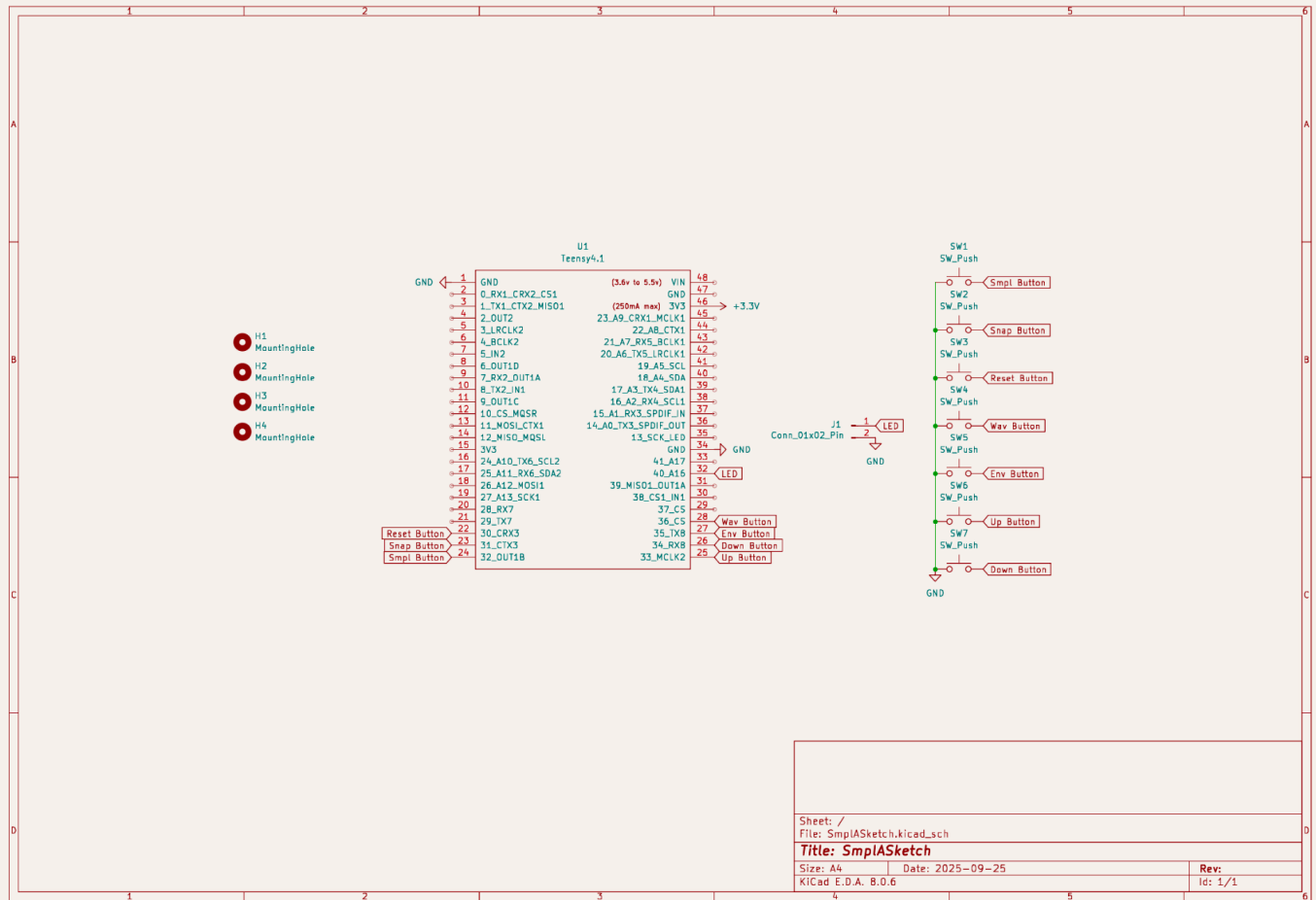
This week I want to entirely be **user testing** by giving it to friends and seeing if I can **find bugs** and break the software to then go and fix it. Then I will write the final report for the class and if I have time, design a website for the device.

Completed Diagram

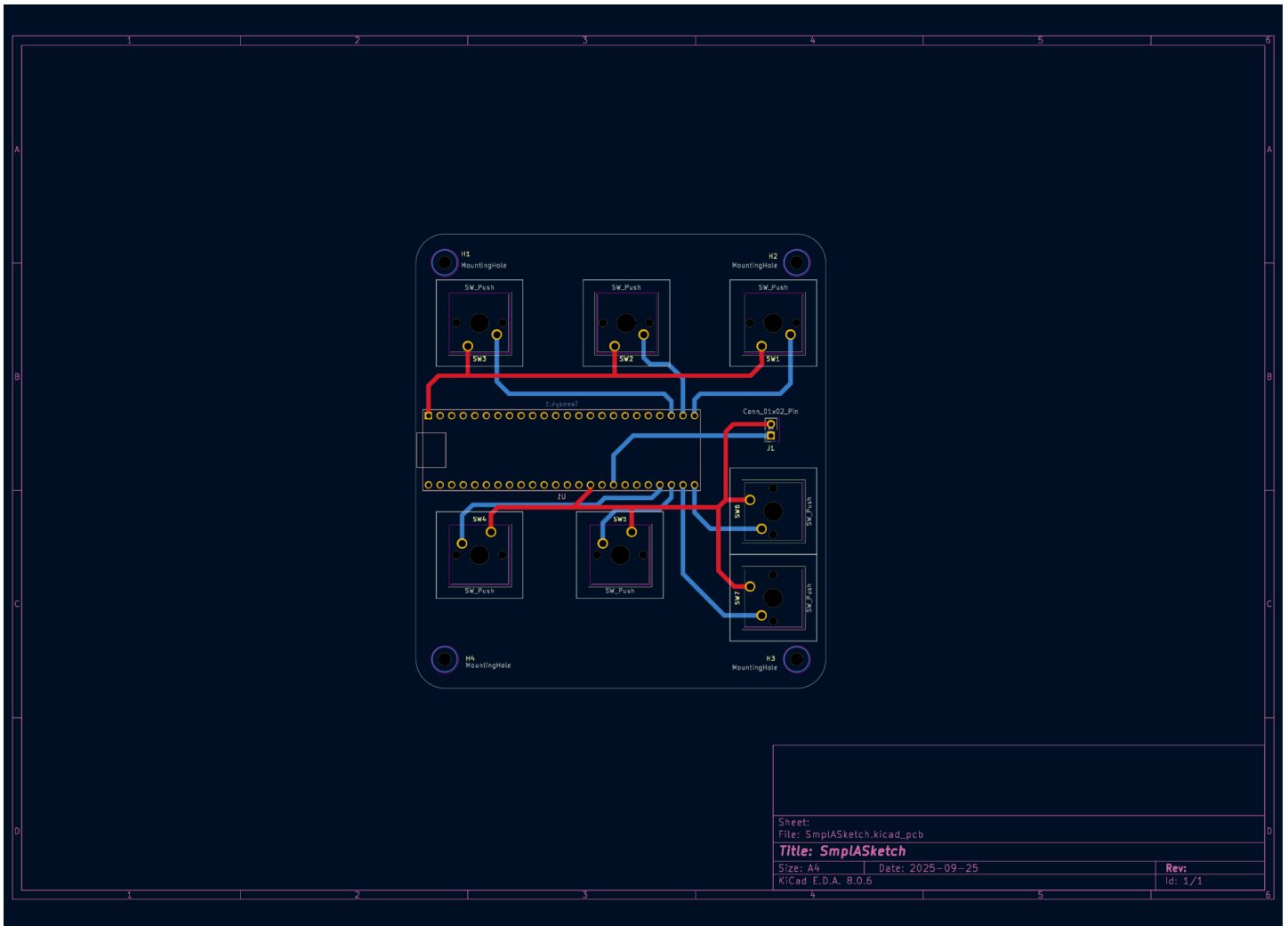


There are **9 user interface elements** on the device including 7 buttons, 1 usb port, and a slot for the user to insert the plexiglass slides. The **plexiglass slides** are drawn on with any whiteboard marker and inserted into the scan slot. For reference of what the internal device looks like scroll down to the section with the current 3D prints. The **Wav and LFO buttons**(previously set as the Envelope storage slot) must be hit first for any other button to work except for the Smpl button. The **Smpl button** plays back whatever the current sample stored is regardless of the mode selected. If the Wav button is selected then Wav mode is active, same for the LFO button and LFO mode. The **reset button** will clear the stored slide capture and replace it with a sine wave in Wav mode and reset to a straight line in LFO mode. The **snap button** will just store the drawing on the slide either in the waveform slot or LFO slot depending on the selected mode. The **up and down arrow keys** increase/decrease pitch in the wav mode and increase/decrease the speed at which the LFO plays back. To swap whether the LFO is controlling frequency or amplitude you can hit both arrow keys at the same time to toggle. Through the use of a USB hub inside the device there is one **USB C cable** to plug the device into the computer and interface with the desktop app.

Schematic



PCB Diagram

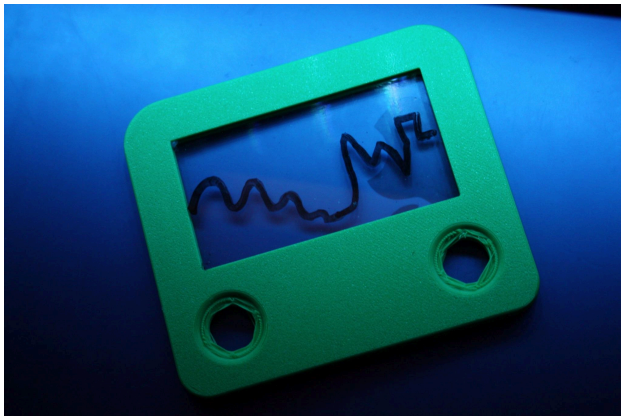
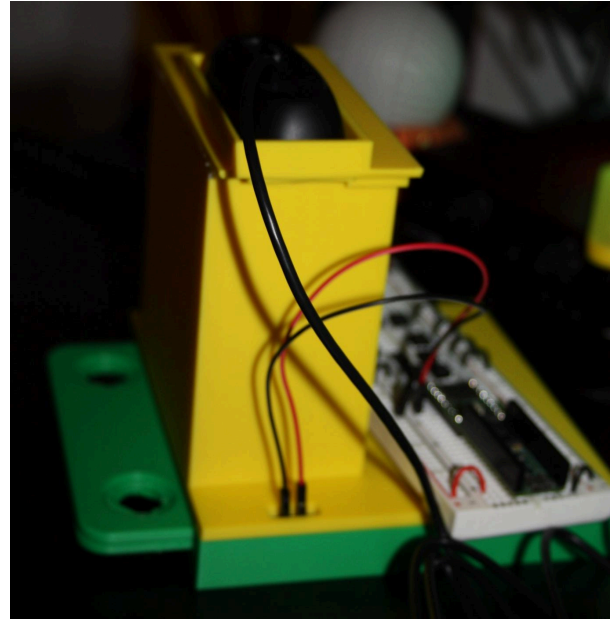
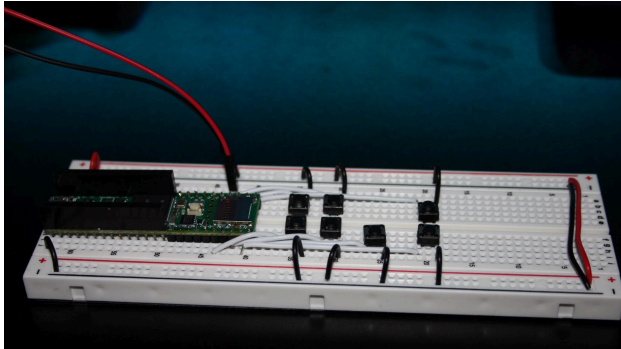


Appendix

Raspberry Pi Headless Setup Guide: [here](#)

GitHub Page: [here](#)*

Current Device Images:



*To view what the code looks like at the time of writing this document without updates added to the github in the future I attached the current version of the code below.

PCB Testing Code

```
int led = 40;      //pcb
int button = 31;   //pcb
int buttonState = HIGH;
int lastButtonState = HIGH;
bool on = false;

void setup() {
    Serial.begin(9600);
    pinMode(led, OUTPUT);
    pinMode(button, INPUT_PULLUP);
}

void loop() {
    if (digitalRead(button) == LOW) {
        on = !on;
        delay(200);
    }
    if (on == true) {
        digitalWrite(led, HIGH);
    } else {
        digitalWrite(led, LOW);
    }
    // checkButton();
    // if (buttonState == LOW && lastButtonState == HIGH) {
    //     digitalWrite(led, HIGH);
    //     delay(6000);
    //     digitalWrite(led, LOW);
    // }
}

void checkButton() {
    lastButtonState = buttonState;
    buttonState = digitalRead(button);
}
```

C++ Code (PointerIHardlyKnowHer)

```
int buttons[7] = { 28, 30, 31, 32, 33, 35, 37 }; //breadboard
//int buttons[7] = { 31, 30, 32, 31, 35, 37, 33 }; //pcb
bool buttonState[7] = { HIGH, HIGH, HIGH, HIGH, HIGH, HIGH, HIGH };
bool lastButtonState[7] = { HIGH, HIGH, HIGH, HIGH, HIGH, HIGH, HIGH };
int led = 36; //breadboard
//int led = 40; //pcb

bool wav = false;
bool env = false;

// pcb Pin 31 is Snap 0 Pin 30 is Reset 1 Pin 32 is Smpl 3 Pin 34 is DownInc 2 Pin 36 is Wav 5 Pin 35 is Env 6 Pin 33 is UpInc 4
// breadboard Pin 28 is Snap 0 Pin 30 is Reset 1 Pin 32 is Smpl 3 Pin 31 is DownInc 2 Pin 35 is Wav 5 Pin 37 is Env 6 Pin 33 is
UpInc 4

void setup() {
    Serial.begin(9600);
    for (int i = 0; i < 7; i++) {
        pinMode(buttons[i], INPUT_PULLUP);
    }
    pinMode(led, OUTPUT);
    delay(1000);
    Keyboard.press(KEY_LEFT_ALT);
    Keyboard.press(KEY_LEFT_SHIFT);
    Keyboard.press('=');
    Keyboard.release(KEY_LEFT_ALT);
    Keyboard.release(KEY_LEFT_SHIFT);
    Keyboard.release('=');
}

void loop() {
    setWav();
    setEnv();
    checkButtons();
    if(buttonState[0] == LOW && lastButtonState[0] == HIGH) {
        snap();
    }
    if(buttonState[1] == LOW && lastButtonState[1] == HIGH) {
        clear();
    }
    if(buttonState[3] == LOW && lastButtonState[3] == HIGH) {
        Keyboard.press(KEY_LEFT_ALT); //smpl
        Keyboard.press('Q');
        Keyboard.press('O');
        delay(100);
        Keyboard.release('O'); //smpl
        Keyboard.release('Q');
        Keyboard.release(KEY_LEFT_ALT);
    }
    if(buttonState[2] == LOW && lastButtonState[2] == HIGH) {
        downInc();
    }
    if(buttonState[4] == LOW && lastButtonState[4] == HIGH) {
        upInc();
    }
}

void checkButtons() {
    for (int i = 0; i < 7; i++) {
        lastButtonState[i] = buttonState[i];
    }
}
```

```

        buttonState[i] = digitalRead(buttons[i]);
    }
}

void snap(){
    digitalWrite(led, HIGH);
    delay(500);
    if (wav == true) {
        Keyboard.press(KEY_LEFT_ALT); //wavsnap
        Keyboard.press(KEY_LEFT_SHIFT);
        Keyboard.press('2');
        delay(100);
        wav = false;
        Keyboard.release(KEY_LEFT_ALT);
        Keyboard.release(KEY_LEFT_SHIFT);
        Keyboard.release('2');
    } else if (env == true) {
        Keyboard.press(KEY_LEFT_ALT); //envsnap
        Keyboard.press('f');
        delay(100);
        env = false;
        Keyboard.release('f');
        Keyboard.release(KEY_LEFT_ALT);
    }
    delay(1000);
    digitalWrite(led, LOW);
}

void clear(){
    if (wav == true) {
        Keyboard.press(KEY_LEFT_ALT); //reset wav
        Keyboard.press('2');
        delay(100);
        wav = false;
        Keyboard.release('2');
        Keyboard.release(KEY_LEFT_ALT);
    } else if (env == true) {
        Keyboard.press(KEY_LEFT_ALT); //reset env
        Keyboard.press(KEY_LEFT_SHIFT);
        Keyboard.press('v');
        delay(100);
        env = false;
        Keyboard.release(KEY_LEFT_ALT);
        Keyboard.release(KEY_LEFT_SHIFT);
        Keyboard.release('v');
    }
}

void upInc(){
    if (wav == true) {
        Keyboard.press(KEY_LEFT_ALT); //inc wav
        Keyboard.press(KEY_LEFT_SHIFT);
        Keyboard.press('8');
        delay(100);
        wav = false;
        Keyboard.release(KEY_LEFT_ALT);
        Keyboard.release(KEY_LEFT_SHIFT);
        Keyboard.release('8');
    } else if (env == true) {
        Keyboard.press(KEY_LEFT_ALT); //inc env
        Keyboard.press(KEY_LEFT_SHIFT);

```

```

    Keyboard.press('7');
    env = false;
    Keyboard.release(KEY_LEFT_ALT);
    Keyboard.release(KEY_LEFT_SHIFT);
    Keyboard.release('7');
}
}

void downInc() {
    if (wav == true) {
        Keyboard.press(KEY_LEFT_ALT); //dec wav
        Keyboard.press(KEY_LEFT_SHIFT);
        Keyboard.press('R');
        delay(100);
        wav = false;
        Keyboard.release(KEY_LEFT_ALT);
        Keyboard.release(KEY_LEFT_SHIFT);
        Keyboard.release('R');
    } else if (env == true) {
        Keyboard.press(KEY_LEFT_ALT); //dec env
        Keyboard.press('t');
        delay(100);
        env = false;
        Keyboard.release(KEY_LEFT_ALT);
        Keyboard.release('t');
    }
}

void setWav() {
    if (buttonState[5] == LOW && lastButtonState[5] == HIGH) {
        wav = true;
        env = false;
        Serial.print("wav");
    }
}

void setEnv() {
    if (buttonState[6] == LOW && lastButtonState[6] == HIGH) {
        env = true;
        wav = false;
        Serial.print("env");
    }
}

```

Processing App Code

smplsketch(main tab)

```
import java.util.Map;
import processing.sound.*;
import g4p_controls.*;

Sound s;

boolean test = false;

//Camera Dimensions Setup
int[] sizeArr = {640, 480}; //fix width to match pi cam(mac cam is 640 480)
int camwidth = sizeArr[0];
int camheight = sizeArr[1];

void setup() {
  //GUI Setup
  size(720, 466, JAVA2D);
  createGUI();
  //img processing setup
  modWavImg = createImage(camwidth, camheight, RGB); // Create image to write other data to
  modEnvImg = createImage(camwidth, camheight, RGB); // Create env image to write other data to
  initImgFromFile("test.png");
  searchForCamera();
  //clear cache
  clearCache();
  //Setup volume control
  s = new Sound(this);
  s.volume(0.2);
  //
}

void draw() {
  background(143, 3, 3);
  checkModel();
  checkForConnection();
}

void keyPressed() { //test true is for keyboard control test false is obscure ascii to talk to teensy
  if ((test == true && key == 'w') || (test == false && key == '€')) {
    wavSnap();
    println("wavSnap");
  } else if ((test == true && key == 'e') || (test == false && key == 'f')) {
    envSnap();
    println("envSnap");
  } else if ((test == true && key == 'f') || (test == false && key == '+')) {
    decEnvMult();
    println("decEnv");
  } else if ((test == true && key == 'f') || (test == false && key == '+')) {
    incEnvMult();
    println("incEnv");
  } else if ((test == true && key == '-') || (test == false && key == '%')) {
    decWavMult();
    println("decWav");
  } else if ((test == true && key == '=') || (test == false && key == '^')) {
    incWavMult();
    println("incWav");
  } else if ((test == true && key == 's') || (test == false && key == 'CE')) {
    smpl();
    println("smpl");
  } else if ((test == true && key == 'r') || (test == false && key == 'TM')) {
    resetWav();
    println("resetWav");
  } else if ((test == true && key == 't') || (test == false && key == 'o')) {
    resetEnv();
    println("resetEnv");
  } else if (key == (char)27) { //Escape key always exits program
    clearCache();
    println("Program Exit");
    exit();
  } else if (key == (char)32) {
    clearActiveSamples();
  }
}
```

audiosampling

```
float volume = 0.2;
float wavMult = 1;
float envMult = 1;
```

```

//Waveform Generation
AudioSample createSample(float[] wav) { //create sample object from float array
    AudioSample sample = new AudioSample(this, wav, 200 * camwidth);
    return sample;
}

void playWav(AudioSample sample) { //takes in sample object and plays it
    sample.rate(0.7 * wavMult); //add pitch shift due to button input
    sample.loop();
}

void playWav(AudioSample sample, float rate) { //takes in sample object and plays it
    sample.rate(rate); //add pitch shift due to button input
    sample.loop();
}

// Envelope Generation
float[] getTimeVals() { //translate envimage processing code into values needed to play the envelope back
    float[] timeVals = new float[3];
    float[] splits = processEnvimage();
    timeVals[0] = (splits[0]/100) * 0.45;
    timeVals[1] = ((splits[1]-splits[0])/100) * 0.45;
    timeVals[2] = ((100-splits[1])/100) * 0.45;
    return timeVals;
}

void playEnv(AudioSample sample) { //takes in a sample object and using the variables plays an envelope
    Env env = new Env(this);
    float[] time = getTimeVals();
    float attackTime = time[0];
    float sustainTime = time[1];
    float releaseTime = time[2];
    env.play(sample, attackTime * envMult, sustainTime * envMult, 1.0, releaseTime);
}

void playDemoEnv(AudioSample sample) { //takes in a demo object and using the variables plays a demo envelope
    Env env = new Env(this);
    float[] time = resetEnv;
    float attackTime = time[0];
    float sustainTime = time[1];
    float releaseTime = time[2];
    env.play(sample, attackTime * envMult, sustainTime * envMult, 1.0, releaseTime);
}

void playSliderEnv(AudioSample sample) { //takes in a demo object and using the variables plays a demo envelope
    Env env = new Env(this);
    int[] time = sliderVals;
    float attackTime = map(time[0], 0, 10, 0, 1);
    float sustainTime = map(time[1], 0, 10, 0, 1);
    float releaseTime = map(time[2], 0, 10, 0, 1);
    env.play(sample, attackTime * envMult, sustainTime * envMult, 1.0, releaseTime);
}

```

buttoncontrol

```

boolean clearWav = true;
boolean wavControl = true;
boolean clearEnv = true;
boolean susMode = false;
boolean sliderMode = false;
int[] sliderVals = new int[3];
boolean spaceHit = false;
ArrayList<AudioSample> endSus = new ArrayList<AudioSample>();

void checkForConnection() {
    if (key == '±') {
        drawConnectionIcon(true);
    } else if (key == 'o') {
        //drawConnectionIcon(false);
    }
}

void wavSnap() {
    takePicture("wav_image");
    wavImg = loadImage("wav_image.jpg");
    clearWav = false;
}

void envSnap() {
    takePicture("env_image");
    envImg = loadImage("env_image.jpg");
    clearEnv = false;
}

```

```

void incWavMult() {
    if (wavMult < 2.0) {
        wavMult += 0.1;
    }
}

void decWavMult() {
    if (wavMult > 0.5) {
        wavMult -= 0.1;
    }
}

void incEnvMult() {
    if (envMult < 2.0) {
        envMult += 0.1;
    }
}

void decEnvMult() {
    if (envMult > 0.5) {
        envMult -= 0.1;
    }
}

void resetWav() {
    wavMult=1;
    clearWav = true;
    filereset.save("wav_image.jpg");
}

void resetEnv() {
    envMult =1;
    clearEnv = true;
    susMode = false;
    sliderMode = false;
    filereset.save("env_image.jpg");
}

void smpl() {
    if (mode != 2.0) {
        float[] wav;
        if (clearWav && wavControl) {
            wav = resetSin;
        } else if (!wavControl) {
            wav = defaultWav;
        } else {
            wav = processWavImage();
        }
        AudioSample smpl = createSample(wav);
        playWav(smpl);
        if (clearEnv) {
            playDemoEnv(smpl);
        } else if (susMode) {
            endSus.add(smpl);
        } else if (sliderMode) {
            playSliderEnv(smpl);
        } else {
            playEnv(smpl);
        }
    } else {
        photowaveformSmpl();
    }
}

void photowaveformSmpl() { //this is very rudimentary and will do more later
    ArrayList<float[]> processed = processMultiImage(imageMode);
    float[] wavR = processed.get(0);
    float[] wavG = processed.get(1);
    float[] wavB = processed.get(2);
    AudioSample smplR = createSample(wavR);
    AudioSample smplG = createSample(wavG);
    AudioSample smplB = createSample(wavB);
    AudioSample body = createSample(resetSin);
    playWav(body, 0.75*wavMult);
    playWav(smplR, 1.0*wavMult);
    playWav(smplG, 1.5*wavMult);
    playWav(smplB, 2.0*wavMult);
    if (clearEnv) {
        playDemoEnv(body);
        playDemoEnv(smplR);
        playDemoEnv(smplG);
        playDemoEnv(smplB);
    } else if (susMode) {
        endSus.add(body);
        endSus.add(smplR);
    }
}

```

```

endSus.add(smplG);
endSus.add(smplB);
} else if (sliderMode) {
  playSliderEnv(body);
  playSliderEnv(smplR);
  playSliderEnv(smplG);
  playSliderEnv(smplB);
} else {
  playEnv(body);
  playEnv(smplR);
  playEnv(smplG);
  playEnv(smplB);
}
}

```

cameracode

```
import processing.video.*;
```

```
Capture cam;
```

```

void searchForCamera() {
  String[] cameras = Capture.list();

  if (cameras.length == 0) {
    println("There are no cameras available for capture.");
    exit();
  } else {
    println("Available cameras:");
    int setcamIndex = -1;
    for (int i = 0; i < cameras.length; i++) {
      println(cameras[i]);
      if (cameras[i].contains("C270")) {
        setcamIndex = i;
      }
    }
    if (setcamIndex != -1 || test) { //allows the program to run eve when device is not connected while testing
      if (test) {
        cam = new Capture(this, 850, 900, "pipeline:avfvideoSrc device-index=0", 30);
      } else {
        cam = new Capture(this, 850, 900, "pipeline:avfvideoSrc device-index="+setcamIndex, 30);
      }
      cam.start();
    } else {
      //display no camera connected dialouge
      wait(5000);
      exit();
    }
  }
}

```

```

void takePicture(String imageName) {
  //delay(1000);
  if (cam.available() == true) {
    cam.read();
  }
  cam.save(imageName+".jpg");
  //image(cam, 0, 0);
}

```

displaymodes

```

float[] resetSin = genSinWav();
float[] resetEnv = genBasicEnv();
float mode = 0.0; //0.0 is Wav Mode 1.0 is Env Mode 2.0 is IMG mode
float prevMode = 0.0;

```

```

void wavMode(float[] wav) { //Draw Screen, draw wav, draw outline
  drawScreen();
  drawWav(wav);
  drawScreenBox();
}

```

```

void envMode(float[] env) { //Draw Screen, draw env, draw outline
  drawScreen();
  drawEnv(env);
  drawScreenBox();
}

```

```

void imgMode() { //Draw Screen, draw wavs(need to add), draw outline
  drawScreen();
  ArrayList<float[]> arrList = processMultiImage(imageMode);
  int counter = 0;
  for (float[] f : arrList) {
    if (counter == 0) {

```

```

    drawWav(f, 255, 0, 0);
  } else if (counter == 1) {
    drawWav(f, 0, 255, 0);
  } else if (counter == 2) {
    drawWav(f, 0, 0, 255);
  }
  counter++;
}
drawScreenBox();
}

void checkMode() { //Would do else ifs but this logic makes the knob smoother
  if (0.0 == mode || prevMode == 0.0) {
    if (clearWav && wavControl) {
      wavMode(resetSin);
    } else if (!wavControl) {
      wavMode(defaultWav);
    } else {
      wavMode(processWavImage());
    }
    prevMode = 0.0;
  }
  if (1.0 == mode || prevMode == 1.0) {
    if (clearEnv) {
      envMode(resetEnv);
    } else if (susMode) {
      envMode(new float[]{0.1, 0.9, 0});
    } else if (sliderMode) {
      float[] mappedSliders = new float[]{map(sliderVals[0], 0, 10, 0, 1), map(sliderVals[1], 0, 10, 0, 1), map(sliderVals[2], 0, 10, 0, 1)};
      envMode(mappedSliders);
    } else {
      envMode(getTimeVals());
    }
    prevMode = 1.0;
  }
  if (2.0 == mode || prevMode == 2.0) {
    imgMode();
    prevMode = 2.0;
  }
}
}

```

drawinghelpers

```

//Data Mapping
float[] mappedFIArr(float[] fArr) { //Map wav values to fit the ui screen
  float[] temp = new float[fArr.length];
  for (int i = 0; i < fArr.length; i++) {
    temp[i] = map(fArr[i], -1, 1, 40, 320);
  }
  return temp;
}

```

```

float[] mapEnv(float[] env) { //Map envelope values to fit the ui screen
  float totalTime = env[0]+env[1]+env[2];
  float attackTime = map(env[0], 0.0, totalTime, 0, 640);
  float sustainTime = map(env[1], 0.0, totalTime, 0, 640);
  float releaseTime = map(env[2], 0.0, totalTime, 0, 640);
  return new float[]{attackTime, sustainTime, releaseTime};
}

```

```

//Draw Functions
void drawScreen() { //draw blank screen
  fill(255);
  stroke(0);
  strokeWeight(2);
  rect(40, 20, 640, 320);
}

```

```

void drawScreenBox() { //draw screen outline
  stroke(0);
  strokeWeight(2);
  noFill();
  rect(40, 20, 640, 320);
}

```

```

void drawWav(float[] wavArr) { //takes wav arr draws it
  stroke(225, 0, 0);
  strokeWeight(2);
  float[] wav = mappedFIArr(wavArr);
  for (int i = 0; i < wav.length-1; i++) {
    line(i+40, wav[i], i+41, wav[i+1]);
  }
}

```

```

void drawWav(float[] wavArr, int r, int g, int b) { //takes wav arr draws it

```

```

stroke(r, g, b);
strokeWeight(2);
float[] wav = mappedFIArr(wavArr);
for (int i = 0; i < wav.length-1; i++) {
  line(i+40, wav[i], i+41, wav[i+1]);
}
}

void drawEnv(float[] envArr) { //takes env arr draws it
stroke(225, 0, 0);
strokeWeight(2);
float[] env = mapEnv(envArr);
line(40, 340, 40+env[0], 200);
line(40+env[0], 200, 40+env[0]+env[1], 200);
line(40+env[0]+env[1], 200, 680, 340);
}

void drawConnectionIcon(boolean bool) { //true for on false for off
redCon.setVisible(!bool);
}

```

gui

```

/* =====
* ===== WARNING =====
* =====
* The code in this tab has been generated from the GUI form
* designer and care should be taken when editing this file.
* Only add/edit code inside the event handlers i.e. only
* use lines between the matching comment tags. e.g.

void myBtnEvents(GButton button) { //_CODE_:button:12356:
// It is safe to enter your event code here
} //_CODE_:button:12356:

* Do not rename this tab!
* =====
*/

public void knob1_turn1(GKnob source, GEvent event) { //_CODE_:modeknob:727750:
setModeKnob(source);
} //_CODE_:modeknob:727750:

public void imgButton1_click1(GImageButton source, GEvent event) { //_CODE_:logo:704728:
} //_CODE_:logo:704728:

public void wavButton_click1(GImageButton source, GEvent event) { //_CODE_:wavButton:649965:
openWavMenu();
} //_CODE_:wavButton:649965:

public void envButton_click1(GImageButton source, GEvent event) { //_CODE_:envButton:585026:
openEnvMenu();
} //_CODE_:envButton:585026:

public void imgButton_click1(GImageButton source, GEvent event) { //_CODE_:imgButton:805293:
} //_CODE_:imgButton:805293:

public void ModeText_click1(GImageButton source, GEvent event) { //_CODE_:ModeText:750609:
} //_CODE_:ModeText:750609:

public void VolText_click1(GImageButton source, GEvent event) { //_CODE_:VolText:249241:
println("imgButton6 - GImageButton >> GEvent." + event + " @ " + millis());
} //_CODE_:VolText:249241:

public void KnobNums_click1(GImageButton source, GEvent event) { //_CODE_:KnobNums:641804:
println("imgButton7 - GImageButton >> GEvent." + event + " @ " + millis());
} //_CODE_:KnobNums:641804:

public void knob2_turn1(GKnob source, GEvent event) { //_CODE_:volknob:873452:
setVolKnob(source);
} //_CODE_:volknob:873452:

public void greenCon_click1(GImageButton source, GEvent event) { //_CODE_:greenCon:794411:
println("imgButton8 - GImageButton >> GEvent." + event + " @ " + millis());
} //_CODE_:greenCon:794411:

public void redCon_click1(GImageButton source, GEvent event) { //_CODE_:redCon:582663:
println("imgButton9 - GImageButton >> GEvent." + event + " @ " + millis());
} //_CODE_:redCon:582663:

public void helpButton_click1(GImageButton source, GEvent event) { //_CODE_:helpButton:816277:
openHelpMenu();
} //_CODE_:helpButton:816277:

// Create all the GUI controls.

```

```

// autogenerated do not edit
public void createGUI(){
    G4P.messagesEnabled(false);
    G4P.setGlobalColorScheme(GCScheme.GOLD_SCHEME);
    G4P.setMouseOverEnabled(false);
    G4P.setDisplayFont("SignPainter", G4P.BOLD, 18);
    G4P.setSliderFont("SignPainter", G4P.BOLD, 15);
    surface.setTitle("Smpl-A-Sketch");
    modeknob = new GKnob(this, 72, 360, 72, 72, 0.8);
    modeknob.setTurnRange(160, 20);
    modeknob.setTurnMode(GKnob.CTRL_HORIZONTAL);
    modeknob.setSensitivity(1);
    modeknob.setShowArcOnly(false);
    modeknob.setOverArcOnly(false);
    modeknob.setIncludeOverBezel(false);
    modeknob.setShowTrack(true);
    modeknob.setLimits(0.0, 0.0, 2.0);
    modeknob.setNbrTicks(3);
    modeknob.setStickToTicks(true);
    modeknob.setShowTicks(true);
    modeknob.setLocalColorScheme(GCScheme.YELLOW_SCHEME);
    modeknob.setOpaque(false);
    modeknob.addEventHandler(this, "knob1_turn1");
    displaylabel = new GLabel(this, 36, 24, 640, 320);
    displaylabel.setTextAlign(GAlign.CENTER, GAlign.MIDDLE);
    displaylabel.setLocalColorScheme(GCScheme.SCHEME_8);
    displaylabel.setOpaque(false);
    logo = new GImageButton(this, 224, 364, new String[] { "BigLogo.png", "BigLogo.png", "BigLogo.png" });
    logo.setLocalColorScheme(GCScheme.BLUE_SCHEME);
    logo.addEventHandler(this, "imgButton1_click1");
    wavButton = new GImageButton(this, 48, 396, 19, 15, new String[] { "wavBg.png", "wavBg.png", "wavBg.png" });
    wavButton.setLocalColorScheme(GCScheme.BLUE_SCHEME);
    wavButton.addEventHandler(this, "wavButton_click1");
    envButton = new GImageButton(this, 96, 348, 25, 10, new String[] { "envBg.png", "envBg.png", "envBg.png" });
    envButton.setLocalColorScheme(GCScheme.BLUE_SCHEME);
    envButton.addEventHandler(this, "envButton_click1");
    imgButton = new GImageButton(this, 150, 396, 19, 16, new String[] { "camBg.png", "camBg.png", "camBg.png" });
    imgButton.setLocalColorScheme(GCScheme.BLUE_SCHEME);
    imgButton.addEventHandler(this, "imgButton_click1");
    ModeText = new GImageButton(this, 84, 433, new String[] { "mode.png", "mode.png", "mode.png" });
    ModeText.setLocalColorScheme(GCScheme.BLUE_SCHEME);
    ModeText.addEventHandler(this, "ModeText_click1");
    VolText = new GImageButton(this, 588, 433, new String[] { "vol.png", "vol.png", "vol.png" });
    VolText.setLocalColorScheme(GCScheme.BLUE_SCHEME);
    VolText.addEventHandler(this, "VolText_click1");
    KnobNums = new GImageButton(this, 556, 350, new String[] { "VolKnob.png", "VolKnob.png", "VolKnob.png" });
    KnobNums.setLocalColorScheme(GCScheme.BLUE_SCHEME);
    KnobNums.addEventHandler(this, "KnobNums_click1");
    volknob = new GKnob(this, 564, 360, 72, 72, 0.8);
    volknob.setTurnRange(110, 70);
    volknob.setTurnMode(GKnob.CTRL_HORIZONTAL);
    volknob.setSensitivity(1);
    volknob.setShowArcOnly(false);
    volknob.setOverArcOnly(false);
    volknob.setIncludeOverBezel(false);
    volknob.setShowTrack(true);
    volknob.setLimits(2.0, 0.0, 10.0);
    volknob.setNbrTicks(11);
    volknob.setStickToTicks(true);
    volknob.setShowTicks(true);
    volknob.setLocalColorScheme(GCScheme.YELLOW_SCHEME);
    volknob.setOpaque(false);
    volknob.addEventHandler(this, "knob2_turn1");
    greenCon = new GImageButton(this, 689, 5, new String[] { "congreen.png", "congreen.png", "congreen.png" });
    greenCon.setLocalColorScheme(GCScheme.BLUE_SCHEME);
    greenCon.addEventHandler(this, "greenCon_click1");
    redCon = new GImageButton(this, 689, 5, new String[] { "conred.png", "conred.png", "conred.png" });
    redCon.setLocalColorScheme(GCScheme.BLUE_SCHEME);
    redCon.addEventHandler(this, "redCon_click1");
    helpButton = new GImageButton(this, 11, 5, new String[] { "help.png", "help.png", "help.png" });
    helpButton.setLocalColorScheme(GCScheme.BLUE_SCHEME);
    helpButton.addEventHandler(this, "helpButton_click1");
}

// Variable declarations
// autogenerated do not edit
GKnob modeknob;
GLabel displaylabel;
GImageButton logo;
GImageButton wavButton;
GImageButton envButton;
GImageButton imgButton;
GImageButton ModeText;
GImageButton VolText;
GImageButton KnobNums;
GKnob volknob;

```

```
GImageButton greenCon;
GImageButton redCon;
GImageButton helpButton;
```

imageprocessing

```
PImage wavImg;
PImage envImg;
PImage modWavImg;
PImage modEnvImg;
```

```
int findAvgY(ArrayList<Integer> list) { // find average of y pixels from drawing
    int avg = 0;
    int size = list.size();
    for (int i = 0; i < size; i++) {
        avg += list.get(i);
    }
    avg = avg/size;
    return avg;
}
```

```
PImage crop(PImage img) { //crops image from camera to get just the slide
PImage cropped = img.get(160, 325, 640, 480);
//cropped.save("cropped.jpg");
return cropped;
}
```

```
float[] processImage(PImage img, PImage modImg, Boolean isWav) { //takes the image from the folder changes all red pixels to green and returns float array of drawn wave
modImg.loadPixels(); //load display pixels
img.loadPixels(); //load images pizels
img = crop(img);
img.filter(THRESHOLD, 0.795); //filters line to black
//img.save("newimg.jpg");
float[] wav = new float[camwidth];
```

```
for (int x = 0; x < camwidth; x++) {
    ArrayList<Integer> readBlackPix = new ArrayList<Integer>(); // create list to hold y values for a single x point
    for (int y = 0; y < camheight; y++) {
        int loc = x + y*camwidth; //calculate pixel location

        // Pull out the 3 color components from pixel
        float r = red(img.pixels[loc]);
        float g = green(img.pixels[loc]);
        float b = blue(img.pixels[loc]);

        // Change red pixels to green to show that it was read
        if (r < 10 && b < 10 && g < 10) {
            r=0;
            b=0;
            g=255;
            readBlackPix.add(y); //add the y value to the list
        }

        // Change displayed pixel to represent coordinates counted
        modImg.pixels[loc] = color(r, g, b);
    }
    if (readBlackPix.isEmpty()) {
        wav[x]=0;
    } else {
        int avg = findAvgY(readBlackPix);
        float avgFl;
        if (isWav) {
            avgFl = map(avg, camheight, 0, -1, 1); //very confused why it's flipped but that's how the numbers work out
        } else {
            avgFl = map(avg, 0, camheight, 0, 1000); //very confused why it's flipped but that's how the numbers work out
        }
        wav[x] = avgFl;
    }
}
modImg.updatePixels();
return wav;
}
```

```
float[] processWavImage() { // feeds in wavImg names to image processing
    return processImage(wavImg, modWavImg, true);
}
```

```
boolean isOnStraight(float f1, float f2, float f3) { //returns a value is surrounded by two like values
    int i1 = Math.round(f1);
    int i2 = Math.round(f2);
    int i3 = Math.round(f3);
    if ((i1 == i2) && (i2 == i3)) {
        return true;
    } else {
        return false;
    }
}
```

```

    }
}

float[] processEnvImage() { //feeds in env imgs to processing function and returns splits
    float[] splits = new float[2];
    float[] scan = processImage(envImg, modEnvImg, false);
    int counter = 1;
    while (!isOnStraight(scan[counter-1], scan[counter], scan[counter+1])) {
        counter++;
    }
    splits[0] = map(counter, 0, camwidth, 0, 100);
    while (isOnStraight(scan[counter-1], scan[counter], scan[counter+1])) {
        counter++;
    }
    if (counter == (scan.length - 1)) {
        break;
    }
}
splits[1] = map(counter, 0, camwidth, 100, 0);
return splits;
}

void printFlArr(float[] wav) { //Print float array
    for (int i = 0; i < wav.length; i++) {
        println(i + " " + wav[i]);
        delay(100);
    }
}

```

imagescanmode

```

PImage imageMode;

void initImgFromFile(String imgName) {
    imageMode = loadImage(imgName);
    imageMode.resize(640, 480);
}

ArrayList<float[]> processMultiImage(PImage img) {
    img.loadPixels(); //load images pixels
    int imgWidth = img.width;
    int imgHeight = img.height;

    float[] redWav = new float[imgWidth];
    float[] greenWav = new float[imgWidth];
    float[] blueWav = new float[imgWidth];

    for (int x = 0; x < imgWidth; x++) {
        Integer redPix = 0;
        Integer greenPix = 0;
        Integer bluePix = 0;

        for (int y = 0; y < imgHeight; y++) {
            int loc = x + y*imgWidth; //calculate pixel location

            // Pull out the 3 color components from pixel
            float r = red(img.pixels[loc]);
            float g = green(img.pixels[loc]);
            float b = blue(img.pixels[loc]);

            // Change red pixels to green to show that it was read
            if (g >= b && g >= r) {
                greenPix++;
            } else if (b >= r && b >= g) {
                bluePix++;
            } else if (r >= b && r >= g) {
                redPix++;
            }
        }
        redWav[x] = map(redPix, 0, imgHeight, 1, -1);
        greenWav[x] = map(greenPix, 0, imgHeight, 1, -1);
        blueWav[x] = map(bluePix, 0, imgHeight, 1, -1);
    }
    ArrayList<float[]> returnThis = new ArrayList<float[]>();
    returnThis.add(redWav);
    returnThis.add(greenWav);
    returnThis.add(blueWav);

    return returnThis;
}

void wait(int time) {
    int start = millis();
    while (millis() <= start+time) {
    }
}

```

```

//come back to this later it kinda works but ultimately unimplemented with this version/ui
//float adjacent440(AudioSample sample) {
// playWav(sample);
// PitchDetector roughMatinee = new PitchDetector(this, 0.8);
// roughMatinee.input(sample);
// float[] pitchAndConfidence = new float[2];
// wait(300);
// float pitch = roughMatinee.analyze(pitchAndConfidence);
// wait(300);
// while (pitchAndConfidence[1] < 0 && pitchAndConfidence[0] != 0.0) {
// pitch = roughMatinee.analyze(pitchAndConfidence);
// wait(10);
// }
// float ogPitch = pitch;
// print(ogPitch);
// float rateAdjust = (440.0 / ogPitch)*0.7;
// playWav(sample, rateAdjust);
// wait(300);
// pitch = roughMatinee.analyze(pitchAndConfidence);
// wait(300);
// while (pitchAndConfidence[1] < 0 || pitchAndConfidence[0] <= ogPitch) {
// playWav(sample, rateAdjust);
// wait(300);
// pitch = roughMatinee.analyze(pitchAndConfidence);
// wait(300);
// }
// println(pitch);
// return rateAdjust;
//}

```

menuqui

// GUI AutoGenSlop For Non Main Windows

```

boolean helpOpen = false;
boolean envOpen = false;
boolean wavOpen = false;
boolean imgOpen = false;

//
// Help Menu
//

synchronized public void help_draw(PApplet appc, GWinData data) {
  appc.background(230);
}

public void closingHelper(GWindow window) { //CODE_.envWindow:608261:
  helpOpen=false;
} //CODE_.envWindow:608261:

GWindow helpMenu;
GImageButton helpMenuImg;

void createHelpGUI() {
  helpMenu = GWindow.getWindow(this, "Help", 50, 0, 240, 480, JAVA2D);
  helpMenu.noLoop();
  helpMenu.setActionOnClose(G4P.CLOSE_WINDOW);
  helpMenu.addDrawHandler(this, "help_draw");
  helpMenu.addOnCloseHandler(this, "closingHelper");
  helpMenuImg = new GImageButton(helpMenu, 0, 0, new String[] { "HelpMenu.png", "HelpMenu.png", "HelpMenu.png" });
  helpMenu.loop();
  helpOpen=true;
}

//
// Envelope Editor
//

synchronized public void win_draw1(PApplet appc, GWinData data) { //CODE_.envWindow:707014:
  appc.background(143, 3, 3);
} //CODE_.envWindow:707014:

public void closingEnv(GWindow window) { //CODE_.envWindow:608261:
  closingEnv();
} //CODE_.envWindow:608261:

public void checkbox1_clicked1(GCheckbox source, GEvent event) { //CODE_.checkbox1:403739:
  deviceControlEnv(event);
} //CODE_.checkbox1:403739:

public void custom_slider1_changel1(GCustomSlider source, GEvent event) { //CODE_.custom_slider1:443042:
  sliderUpdate(source, event, 0);
} //CODE_.custom_slider1:443042:

```

```

public void custom_slider2_change1(GCustomSlider source, GEvent event) { //_CODE_:custom_slider2:875688:
    sliderUpdate(source, event, 1);
} //_CODE_:custom_slider2:875688:

public void custom_slider3_change1(GCustomSlider source, GEvent event) { //_CODE_:custom_slider3:577796:
    sliderUpdate(source, event, 2);
} //_CODE_:custom_slider3:577796:

public void option1_clicked1(GOption source, GEvent event) { //_CODE_:sustain:236083:
    sustainMode(event);
} //_CODE_:sustain:236083:

public void option2_clicked1(GOption source, GEvent event) { //_CODE_:sliderRatios:316247:
    sliderModeEnv(event);
} //_CODE_:sliderRatios:316247:

public void option1_clicked2(GOption source, GEvent event) { //_CODE_:option1:402650:
    envModeReset(event);
} //_CODE_:option1:402650:

public void imgButton1_click2(GImageButton source, GEvent event) { //_CODE_:hideAllEnv:656167:
} //_CODE_:hideAllEnv:656167:

public void imgButton2_click1(GImageButton source, GEvent event) { //_CODE_:hideSlidersEnv:513038:
} //_CODE_:hideSlidersEnv:513038:

GWindow envWindow;
GCheckbox checkbox1;
GCustomSlider custom_slider1;
GCustomSlider custom_slider2;
GCustomSlider custom_slider3;
GLabel attack;
GLabel susText;
GLabel relText;
GToggleGroup togGroup1;
GOption sustain;
GOption sliderRatios;
GOption option1;
GImageButton hideAllEnv;
GImageButton hideSlidersEnv;

void createEnvGUI() {
    envWindow = GWindow.getWindow(this, "Envelope Editor", 25, 650, 290, 241, JAVA2D);
    envWindow.noLoop();
    envWindow.setActionOnClose(G4P.CLOSE_WINDOW);
    envWindow.addDrawHandler(this, "win_draw1");
    envWindow.addOnCloseHandler(this, "closingEnv");
    checkbox1 = new GCheckbox(envWindow, 166, 22, 108, 22);
    checkbox1.setIconAlign(GAlign.LEFT, GAlign.MIDDLE);
    checkbox1.setText("Device Control");
    checkbox1.setLocalColorScheme(GCScheme.SHEME_8);
    checkbox1.setOpaque(false);
    checkbox1.addEventHandler(this, "checkbox1_clicked1");
    checkbox1.setSelected(true);
    custom_slider1 = new GCustomSlider(envWindow, 10, 69, 266, 50, "red_yellow18px");
    custom_slider1.setShowLimits(true);
    custom_slider1.setLimits(5, 0, 10);
    custom_slider1.setNbrTicks(11);
    custom_slider1.setStickToTicks(true);
    custom_slider1.setShowTicks(true);
    custom_slider1.setNumberFormat(G4P.INTEGER, 0);
    custom_slider1.setLocalColorScheme(GCScheme.SHEME_8);
    custom_slider1.setOpaque(false);
    custom_slider1.addEventHandler(this, "custom_slider1_change1");
    custom_slider2 = new GCustomSlider(envWindow, 10, 127, 266, 50, "red_yellow18px");
    custom_slider2.setShowLimits(true);
    custom_slider2.setLimits(5, 0, 10);
    custom_slider2.setNbrTicks(11);
    custom_slider2.setStickToTicks(true);
    custom_slider2.setShowTicks(true);
    custom_slider2.setNumberFormat(G4P.INTEGER, 0);
    custom_slider2.setLocalColorScheme(GCScheme.SHEME_8);
    custom_slider2.setOpaque(false);
    custom_slider2.addEventHandler(this, "custom_slider2_change1");
    custom_slider3 = new GCustomSlider(envWindow, 10, 183, 266, 50, "red_yellow18px");
    custom_slider3.setShowLimits(true);
    custom_slider3.setLimits(5, 0, 10);
    custom_slider3.setNbrTicks(11);
    custom_slider3.setStickToTicks(true);
    custom_slider3.setShowTicks(true);
    custom_slider3.setNumberFormat(G4P.INTEGER, 0);
    custom_slider3.setLocalColorScheme(GCScheme.SHEME_8);
    custom_slider3.setOpaque(false);
    custom_slider3.addEventHandler(this, "custom_slider3_change1");
    attack = new GLabel(envWindow, 10, 110, 265, 20);
    attack.setTextAlign(GAlign.CENTER, GAlign.MIDDLE);

```

```

attack.setText("Attack");
attack.setLocalColorScheme(GCScheme.SCHEME_15);
attack.setOpaque(false);
susText = new JLabel(envWindow, 10, 167, 266, 20);
susText.setTextAlign(GAlign.CENTER, GAlign.MIDDLE);
susText.setText("Sustain");
susText.setLocalColorScheme(GCScheme.SCHEME_15);
susText.setOpaque(false);
relText = new JLabel(envWindow, 8, 218, 274, 20);
relText.setTextAlign(GAlign.CENTER, GAlign.MIDDLE);
relText.setText("Release");
relText.setLocalColorScheme(GCScheme.SCHEME_15);
relText.setOpaque(false);
togGroup1 = new GToggleGroup();
sustain = new GOption(envWindow, 15, 14, 120, 20);
sustain.setIconAlign(GAlign.LEFT, GAlign.MIDDLE);
sustain.setText("Sustain Mode");
sustain.setLocalColorScheme(GCScheme.SCHEME_13);
sustain.setOpaque(false);
sustain.addEventHandler(this, "option1_clicked1");
sliderRatios = new GOption(envWindow, 15, 37, 120, 20);
sliderRatios.setIconAlign(GAlign.LEFT, GAlign.MIDDLE);
sliderRatios.setText("Use Slider Ratios");
sliderRatios.setLocalColorScheme(GCScheme.SCHEME_10);
sliderRatios.setOpaque(false);
sliderRatios.addEventHandler(this, "option2_clicked1");
option1 = new GOption(envWindow, 85, 400, 120, 20);
option1.setIconAlign(GAlign.LEFT, GAlign.MIDDLE);
option1.setText("option text");
option1.setOpaque(false);
option1.addEventHandler(this, "option1_clicked2");
togGroup1.addControl(sustain);
togGroup1.addControl(sliderRatios);
togGroup1.addControl(option1);
option1.setSelected(true);
hideAllEnv = new GImageButton(envWindow, 0, 0, 289, 241, new String[] { "EnvHideNonDev.png", "EnvHideNonDev.png", "EnvHideNonDev.png" });
hideAllEnv.addEventHandler(this, "imgButton1_click2");
hideSlidersEnv = new GImageButton(envWindow, 0, 0, 289, 241, new String[] { "EnvHideSliders.png", "EnvHideSliders.png", "EnvHideSliders.png" });
hideSlidersEnv.addEventHandler(this, "imgButton2_click1");
envWindow.loop();
sustain.setVisible(false);
sliderRatios.setVisible(false);
envOpen = true;
}

//
// Waveform Editor
//
synchronized public void waveditordraw(PApplet appc, GWinData data) { //_CODE_:wavWindow:973281:
    appc.background(143, 3, 3);
} //_CODE_:wavWindow:973281:

public void closingWav(GWindow window) { //_CODE_:envWindow:608261:
    closingWav();
}

public void wavdevice1_clicked1(GOption source, GEvent event) { //_CODE_:wavdevice1:252560:
    if (event == GEvent.SELECTED) {
        tonalControl();
    }
} //_CODE_:wavdevice1:252560:

public void pitchmode_clicked1(GOption source, GEvent event) { //_CODE_:pitchmode:772210:
    if (event == GEvent.SELECTED) {
        atonalControl();
    }
} //_CODE_:pitchmode:772210:

public void pitch_changel1(GCustomSlider source, GEvent event) { //_CODE_:pitch:634832:
    freqSlider(source, event);
} //_CODE_:pitch:634832:

public void sinwave_clicked1(GOption source, GEvent event) { //_CODE_:sinwave:493274:
    if (event == GEvent.SELECTED) {
        defaultSin();
    }
} //_CODE_:sinwave:493274:

public void squarewav_clicked1(GOption source, GEvent event) { //_CODE_:squarewav:868981:
    if (event == GEvent.SELECTED) {
        defaultSqr();
    }
} //_CODE_:squarewav:868981:

public void trianglewav_clicked1(GOption source, GEvent event) { //_CODE_:trianglewav:982005:
    if (event == GEvent.SELECTED) {

```

```

        defaultTri();
    }
} //CODE_:trianglewav:982005:

public void sawwav_clicked1(GOption source, GEvent event) { //CODE_:sawwav:860623:
    if (event == GEvent.SELECTED) {
        defaultSaw();
    }
} //CODE_:sawwav:860623:

public void rampwav_clicked1(GOption source, GEvent event) { //CODE_:rampwav:647942:
    if (event == GEvent.SELECTED) {
        defaultRamp();
    }
} //CODE_:rampwav:647942:

public void stepwav_clicked1(GOption source, GEvent event) { //CODE_:stepwav:750740:
    if (event == GEvent.SELECTED) {
        defaultStep();
    }
} //CODE_:stepwav:750740:

public void nowav_clicked1(GOption source, GEvent event) { //CODE_:nowav:484575:
    println("nowav - GOption >> GEvent." + event + " @ " + millis());
} //CODE_:nowav:484575:

public void wavcontrol_clicked1(GOption source, GEvent event) { //CODE_:wavcontrol:638305:
    if (event == GEvent.SELECTED) {
        wavDeviceControlOn();
    }
} //CODE_:wavcontrol:638305:

public void defaultwavs_clicked1(GOption source, GEvent event) { //CODE_:defaultwavs:227314:
    if (event == GEvent.SELECTED) {
        wavDeviceControlOff();
    }
} //CODE_:defaultwavs:227314:

public void reddefaultwav_click1(GImageButton source, GEvent event) { //CODE_:reddefaultwav:743207:
    println("reddefaultwav - GImageButton >> GEvent." + event + " @ " + millis());
} //CODE_:reddefaultwav:743207:

public void greystuff_click2(GImageButton source, GEvent event) { //CODE_:greystuff:598096:
    println("imgButton1 - GImageButton >> GEvent." + event + " @ " + millis());
} //CODE_:greystuff:598096:

public void wavlock_click2(GImageButton source, GEvent event) { //CODE_:wavlock:363349:
    println("wavlock - GImageButton >> GEvent." + event + " @ " + millis());
} //CODE_:wavlock:363349:

public void sliderwavhider_click2(GImageButton source, GEvent event) { //CODE_:sliderwavhider:617266:
    println("sliderwavhider - GImageButton >> GEvent." + event + " @ " + millis());
} //CODE_:sliderwavhider:617266:

void createWavGUI() {
    wavWindow = GWindow.getWindow(this, "Waveform Editor", 1150, 500, 390, 300, JAVA2D);
    wavWindow.noLoop();
    wavWindow.setActionOnClose(G4P.CLOSE_WINDOW);
    wavWindow.addDrawHandler(this, "waveditordraw");
    wavWindow.addOnCloseHandler(this, "closingWav");
    waveditoroptions = new GToggleGroup();
    wavdevice1 = new GOption(wavWindow, 215, 15, 140, 20);
    wavdevice1.setIconAlign(GAlign.LEFT, GAlign.MIDDLE);
    wavdevice1.setText("Tonal Device Control");
    wavdevice1.setLocalColorScheme(GCScheme.SCHEME_8);
    wavdevice1.setOpaque(false);
    wavdevice1.addEventHandler(this, "wavdevice1_clicked1");
    pitchmode = new GOption(wavWindow, 30, 15, 170, 20);
    pitchmode.setIconAlign(GAlign.LEFT, GAlign.MIDDLE);
    pitchmode.setText("Atonal Frequency Control");
    pitchmode.setLocalColorScheme(GCScheme.SCHEME_9);
    pitchmode.setOpaque(false);
    pitchmode.addEventHandler(this, "pitchmode_clicked1");
    waveditoroptions.addControl(wavdevice1);
    wavdevice1.setSelected(true);
    waveditoroptions.addControl(pitchmode);
    pitch = new GCustomSlider(wavWindow, 30, 44, 328, 40, "red_yellow18px");
    pitch.setLimits(1, 0, 2);
    pitch.setNumberFormat(G4P.INTEGER, 0);
    pitch.setLocalColorScheme(GCScheme.SCHEME_8);
    pitch.setOpaque(false);
    pitch.addEventHandler(this, "pitch_change1");
    wavgroup = new GToggleGroup();
    sinwave = new GOption(wavWindow, 25, 184, 120, 20);
    sinwave.setIconAlign(GAlign.LEFT, GAlign.MIDDLE);

```

```

sinwave.setOpaque(false);
sinwave.addEventHandler(this, "sinwave_clicked1");
squarewav = new GOption(wavWindow, 25, 236, 120, 20);
squarewav.setIconAlign(GAlign.LEFT, GAlign.MIDDLE);
squarewav.setOpaque(false);
squarewav.addEventHandler(this, "squarewav_clicked1");
trianglewav = new GOption(wavWindow, 145, 184, 120, 20);
trianglewav.setIconAlign(GAlign.LEFT, GAlign.MIDDLE);
trianglewav.setOpaque(false);
trianglewav.addEventHandler(this, "trianglewav_clicked1");
sawwav = new GOption(wavWindow, 145, 236, 120, 20);
sawwav.setIconAlign(GAlign.LEFT, GAlign.MIDDLE);
sawwav.setOpaque(false);
sawwav.addEventHandler(this, "sawwav_clicked1");
rampwav = new GOption(wavWindow, 265, 184, 100, 20);
rampwav.setIconAlign(GAlign.LEFT, GAlign.MIDDLE);
rampwav.setOpaque(false);
rampwav.addEventHandler(this, "rampwav_clicked1");
stepwav = new GOption(wavWindow, 265, 236, 100, 20);
stepwav.setIconAlign(GAlign.LEFT, GAlign.MIDDLE);
stepwav.setOpaque(false);
stepwav.addEventHandler(this, "stepwav_clicked1");
nowav = new GOption(wavWindow, 540, 235, 120, 20);
nowav.setIconAlign(GAlign.LEFT, GAlign.MIDDLE);
nowav.setText("option text");
nowav.setOpaque(false);
nowav.addEventHandler(this, "nowav_clicked1");
wavgroup.addControl(sinwave);
wavgroup.addControl(squarewav);
wavgroup.addControl(trianglewav);
wavgroup.addControl(sawwav);
wavgroup.addControl(rampwav);
wavgroup.addControl(stepwav);
wavgroup.addControl(nowav);
nowav.setSelected(true);
Wavdevicegroup = new GToggleGroup();
wavcontrol = new GOption(wavWindow, 20, 100, 120, 20);
wavcontrol.setIconAlign(GAlign.LEFT, GAlign.MIDDLE);
wavcontrol.setText("Device Control");
wavcontrol.setLocalColorScheme(GCScheme.SCHEME_8);
wavcontrol.setOpaque(false);
wavcontrol.addEventHandler(this, "wavcontrol_clicked1");
defaultwavs = new GOption(wavWindow, 20, 127, 180, 20);
defaultwavs.setIconAlign(GAlign.LEFT, GAlign.MIDDLE);
defaultwavs.setText("Use Default Waveforms");
defaultwavs.setLocalColorScheme(GCScheme.SCHEME_14);
defaultwavs.setOpaque(false);
defaultwavs.addEventHandler(this, "defaultwavs_clicked1");
Wavdevicegroup.addControl(wavcontrol);
wavcontrol.setSelected(true);
Wavdevicegroup.addControl(defaultwavs);
reddefaultwav = new GImageButton(wavWindow, 43, 170, new String[] { "redstuff.png", "redstuff.png", "redstuff.png" });
reddefaultwav.addEventHandler(this, "reddefaultwav_click1");
greystuff = new GImageButton(wavWindow, 43, 170, new String[] { "greyed stuff.png", "greyed stuff.png", "greyed stuff.png" });
greystuff.addEventHandler(this, "greystuff_click2");
wavlock = new GImageButton(wavWindow, 0, 0, 390, 300, new String[] { "WavLockOverlay.png", "WavLockOverlay.png", "WavLockOverlay.png" });
wavlock.addEventHandler(this, "wavlock_click2");
sliderwavhider = new GImageButton(wavWindow, 0, 0, 390, 88, new String[] { "SliderWav.png", "SliderWav.png", "SliderWav.png" });
sliderwavhider.addEventHandler(this, "sliderwavhider_click2");
wavWindow.loop();
wavOpen=true;
}

```

```

GWindow wavWindow;
GToggleGroup waveditoroptions;
GOption wavdevice;
GOption pitchmode;
GCustomSlider pitch;
GToggleGroup wavgroup;
GOption sinwave;
GOption squarewav;
GOption trianglewav;
GOption sawwav;
GOption rampwav;
GOption stepwav;
GOption nowav;
GToggleGroup Wavdevicegroup;
GOption wavcontrol;
GOption defaultwavs;
GImageButton reddefaultwav;
GImageButton greystuff;
GImageButton wavlock;
GImageButton sliderwavhider;

```

presets

```
PImage filereset = createImage(camwidth, camheight, RGB);
```

```
float[] genSinWav() { //create sin wav arr  
float[] sinewave = new float[640];
```

```
for (int i = 0; i < 640; i++) {  
    sinewave[i] = -sin(TWO_PI*i/640);  
}  
return sinewave;  
}
```

```
float[] genBasicEnv() { //create reset env  
float attackTime = 0.1;  
float sustainTime = 0.5;  
float releaseTime = 0.4;  
return new float[] {attackTime, sustainTime, releaseTime};  
}
```

```
void clearCache() {  
    resetWav();  
    resetEnv();  
    println("Image Cache Cleared");  
}
```

```
void clearActiveSamples() {  
    for (AudioSample s : endSus) {  
        s.stop();  
    }  
    endSus = new ArrayList<AudioSample>();  
}
```

userinput

```
//Waveform Editor
```

```
void openWavMenu() {  
    if (wavOpen == false) {  
        createWavGUI();  
    }  
}
```

```
void closingWav() {  
    wavOpen = false;  
}
```

```
float[] defaultWav = resetSin;
```

```
void wavDeviceControlOn() {  
    wavControl = true;  
    nowav.setSelected(true);  
    greystuff.setVisible(true);  
    wavlock.setVisible(true);  
}
```

```
void wavDeviceControlOff() {  
    wavControl = false;  
    sinwave.setSelected(true);  
    greystuff.setVisible(false);  
    wavlock.setVisible(false);  
}
```

```
void tonalControl() {  
    sliderwavhider.setVisible(true);  
    pitch.setLimits(1, 0, 2); // fix limits  
    wavMult=1;  
}
```

```
void atonalControl() {  
    sliderwavhider.setVisible(false);  
    pitch.setLimits(1, 0, 2); // fix limits  
    wavMult=1;  
}
```

```
void freqSlider(GCustomSlider source, GEvent event) {  
    if (event == GEvent.RELEASED) {  
        wavMult = source.getValueF();  
    }  
}
```

```
//Wav Generation Functions
```

```
void defaultSin() {  
    defaultWav = resetSin;  
}
```

```

void defaultSqr() {
    float[] sqrwav = new float[640];

    for (int i = 0; i < 640; i++) {
        if (i < 320) {
            sqrwav[i] = -1;
        } else {
            sqrwav[i] = 1;
        }
    }
    defaultWav=sqrwav;
}

void defaultTri() {
    float[] triwav = new float[640];
    int counter = 0;
    for (int i = 0; i < 640; i++) {
        if (i == 0) {
            counter = 0;
        } else if (i < 160) {
            counter +=1;
        } else if (i < 480) {
            counter -=1;
            triwav[i] = counter;
        } else {
            counter +=1;
        }
        triwav[i] = map(counter, -159, 159, 1, -1);
    }
    defaultWav=triwav;
}

void defaultSaw() {
    float[] sawwav = new float[640];
    int counter = 320;
    for (int i = 0; i < 640; i++) {
        counter -=1;
        sawwav[i] = map(counter, -319, 319, 1, -1);
    }
    defaultWav = sawwav;
}

void defaultRamp() {
    float[] rampwav = new float[640];
    int counter = -320;
    for (int i = 0; i < 640; i++) {
        counter +=1;
        rampwav[i] = map(counter, -319, 319, 1, -1);
    }
    defaultWav=rampwav;
}

void defaultStep() {
    float[] stepwav = new float[640];
    int counter = 0;
    int step = 0;
    for (int i = 0; i < 640; i++) {
        if (step != 16) {
            step++;
        } else {
            counter++;
            step=0;
        }
        stepwav[i] = map(counter, 0, 39, 1, -1);
    }
    defaultWav=stepwav;
}

// Envelope Editor
void openEnvMenu() {
    if (envOpen == false) {
        createEnvGUI();
    }
}

void closingEnv() {
    susMode = false;
    sliderMode = false;
    clearEnv = true;
    envOpen=false;
}

void deviceControlEnv(GEvent event) {
    if (event == GEvent.DESELECTED) {
        hideAllEnv.setVisible(false);
    }
}

```

```

option1.setSelected(true);
sustain.setVisible(true);
sliderRatios.setVisible(true);
} else if (event == GEvent.SELECTED) {
    sustain.setVisible(false);
    sliderRatios.setVisible(false);
    hideAllEnv.setVisible(true);
    option1.setSelected(true);
    hideSlidersEnv.setVisible(true);
    custom_slider1.setLimits(5, 0, 10);
    custom_slider2.setLimits(5, 0, 10);
    custom_slider3.setLimits(5, 0, 10);
    susMode = false;
    sliderMode = false;
    clearEnv = true;
}
}

void sustainMode(GEvent event) {
    if (event == GEvent.SELECTED) {
        hideSlidersEnv.setVisible(true);
        custom_slider1.setLimits(5, 0, 10);
        custom_slider2.setLimits(5, 0, 10);
        custom_slider3.setLimits(5, 0, 10);
        susMode = true;
        clearEnv = false;
        sliderMode = false;
    }
}

void sliderModeEnv(GEvent event) {
    if (event == GEvent.SELECTED) {
        hideSlidersEnv.setVisible(false);
        custom_slider1.setLimits(5, 0, 10);
        custom_slider2.setLimits(5, 0, 10);
        custom_slider3.setLimits(5, 0, 10);
        sliderVals = new int[]{5, 5, 5};
        susMode = false;
        clearEnv = false;
        sliderMode = true;
    }
}

void envModeReset(GEvent event) {
    if (event == GEvent.SELECTED) {
        hideSlidersEnv.setVisible(true);
        custom_slider1.setLimits(5, 0, 10);
        custom_slider2.setLimits(5, 0, 10);
        custom_slider3.setLimits(5, 0, 10);
        susMode = false;
        sliderMode = false;
        clearEnv = true;
    }
}

void sliderUpdate(GCustomSlider source, GEvent event, int index) {
    if (event == GEvent.RELEASED) {
        sliderVals[index] = source.getValue();
    }
}

//Help Menu
void openHelpMenu() {
    if (helpOpen == false) {
        createHelpGUI();
    }
}

//Main Window
void setModeKnob(GKnob source) {
    mode = source.getValueF();
}

void setVolKnob(GKnob source) {
    volume = Math.round(source.getValue()) * 0.1; //Round volume
    s.volume(volume);
}

```